

StrayWatch: Solution Ideation

Rescue dispatch for stray animals on Indian roads, run by AI agents and people in one team chat

Hackathon	Flip the Switch (WeMakeDevs × DevTools Academy × Switch AI), Nov 4–7, 2026
Track	Track 01: Best Business Workflow
Builder	Meenakshi Satpathy, BIT Mesra, Ranchi (solo)
Code	github.com/meenakshisatpathy20-del/SWITCH-AI

1. The theme

The challenge: turn your solo agent into a teammate. Build a workflow where an AI agent and the people in a team chat get a real task done together, connected by Switch: open-source infrastructure that turns every chat channel into a room that agents join like any other teammate.

2. Why I chose this problem

On my way to college in Ranchi, I saw an injured dog lying on the road while vehicles kept passing it. Nobody called anyone, because nobody knew who to call. That moment is why I started StrayWatch.

3. The problem

Stray cattle and dogs on busy roads block traffic, cause accidents and are themselves injured or killed. In 2024, Odisha recorded 437 deaths from 903 animal–vehicle collisions; Haryana has seen 900+ deaths in five years from stray-cattle accidents; in Bengaluru a biker died after being chased by stray dogs.

Police, municipal teams, NGOs and volunteers all exist. The real problem is that help arrives too late and works blind:

- Reports arrive as scattered calls and WhatsApp forwards, often with no exact location
- The same animal is reported again and again by different people

- Nobody knows who is handling a case, so cases get lost in the scroll
- Nobody records where and when it happens, so the same road is dangerous again tomorrow at 6 PM

This is a team coordination problem inside a chat app, which is exactly what Switch is built for.

4. The idea

StrayWatch is ambulance dispatch for animals on roads. One Telegram group becomes a Switch room. Citizens and responders are in it as people; three AI agents are in it as teammates. A citizen only posts what they see. The agents and responders do the rest, in the open, until the case is closed with proof.

Step	Who	What happens
1. Triage	sw-triage (agent)	Reads the report (text/photo, Hindi or English), asks for the location if missing, files a case, sets urgency HIGH / CHECK / LOW, catches duplicates
2. Dispatch	sw-dispatch (agent)	Tags the right responder: traffic police for blocked lanes, municipal team for cattle, NGO for injuries, ABC team for aggressive dogs
3. Accept	Responder (person)	Replies "accept SW-0046", so everyone sees who owns the case
4. Escalate	sw-watch (custom agent)	If nobody accepts within 10 minutes, the duty coordinator is tagged
5. Close with proof	Responder + sw-dispatch	The responder posts a photo; dispatch looks at it and closes the case only if the animal is helped and the road is clear
6. Prevent	sw-hotspot (agent)	Turns the case history into action: "Ratu Road, 6–7 PM, cattle: send the cattle team at 5 PM, before the rush"

5. Why Switch is central

Remove Switch and StrayWatch stops working. The Switch room is the dispatch system:

- Messaging bridge: people report and respond from Telegram, the app they already use; nobody installs anything
- Agents as teammates: three agents with distinct jobs post in the same conversation as humans
- Agent-to-agent handoff in the open: triage hands cases to dispatch by name, so people can see and correct every step
- Main agent + "anyone" addressing: citizens don't need agent names or a Switch account
- Attachments reach agents: this is what makes "close only with photo proof" possible
- Agent protocol: a custom Python agent (sw-watch) keeps the clock, because Switch has no scheduler
- Templates: the whole workflow is one YAML file another city can reuse

6. Design principles

- Code decides urgency, AI extracts facts. Rules live in a tested script, so the same report always gets the same urgency
- No invented data. Agents never guess a location, a case id or a handle, and refuse to recreate a missing case from memory
- Animal welfare first. Aggressive dogs go to ABC (sterilisation & vaccination) teams, never "removal"; reporters are told not to approach injured animals

- Honest data. Demo history is clearly labelled as simulated, and kept separate from live cases

7. Who uses it

User	Pain today	Why they use StrayWatch
NGOs, vets, rescuers	Cases arrive scattered, without a location	One clean, located, urgency-sorted queue
Feeders, caring citizens	They post, but nothing happens	Their report reaches a responder and they see the result
Traffic control	Hears late, from jams and complaints	Only real, high-impact alerts
Municipal teams	No data on where or when	Weekly hotspot list of roads and times

8. What is built (working prototype)

- Three Switch agents (Claude Code) working in a live Telegram room; sw-watch, a custom escalation agent built on the Switch agent protocol, is included (dispatch also runs the same escalation check whenever it is addressed)
- Case ledger with deterministic urgency, routing, duplicate merge, accept / close-with-proof and escalation
- Hotspot report with recommended actions, on 45 simulated cases on real Ranchi roads
- 8 automated tests, MIT licence, step-by-step setup in the README, and a demo video

9. Impact we would measure in a pilot

- Time from report to a responder accepting, and to the road being cleared
- Share of injured-animal cases closed with photo proof
- Fewer repeat cases on hotspot roads after teams act before the rush

10. Roadmap

- Camera agent: YOLOv8 on existing traffic cameras flags animals that stay inside a lane or lie still, and posts them into the room like any citizen report
- Separate public Reports and private Responders rooms, linked in Switch, so exact locations go only to verified responders
- WhatsApp intake and Hindi / Kannada voice notes
- Pilot in Ranchi, then scale to Bengaluru through municipal and Smart City road-safety programmes

Help the animal, clear the road.

Sources for statistics: The Statesman (Odisha 2024 data), PTI / Business Standard (Haryana), India TV and Deccan Herald (Bengaluru).